

Question Index - All 40 Questions

Q#	Question
Q1	Chef basics - architecture, workstation, server, nodes, recipes
Q2	Chef resources - package, service, file, template, execute
Q3	Chef recipes and cookbooks - structure, metadata, dependencies
Q4	Chef attributes - default, normal, override, automatic, precedence
Q5	Chef templates - ERB, variables, conditionals, loops
Q6	Chef run-list - ordering, roles, environments
Q7	Chef data bags - storing shared configuration, encrypted data bags
Q8	Chef knife - CLI tool, bootstrap, node management, cookbook upload
Q9	Puppet basics - architecture, master, agent, catalog, manifests
Q10	Puppet resources - package, service, file, user, exec, notify
Q11	Puppet manifests and modules - structure, init.pp, params.pp
Q12	Puppet classes and defined types - reusable configurations
Q13	Puppet Hiera - external data, hierarchy, automatic lookup
Q14	Puppet Facter - system facts, custom facts, conditional logic
Q15	Chef vs Puppet vs Ansible - architecture, language, comparison
Q16	Chef environments - dev/staging/prod, version pinning
Q17	Chef roles - abstraction, run-lists, attribute overrides
Q18	Chef Berkshelf - dependency management, Berksfile, vendoring
Q19	Chef InSpec - compliance as code, profiles, controls, audit
Q20	Chef Test Kitchen - testing cookbooks, drivers, verifiers
Q21	Chef Policyfile - modern workflow, replacing roles and environments
Q22	Chef Habitat - application automation, plans, supervisors
Q23	Chef Infra Client - solo mode, zero mode, local development
Q24	Puppet environments - directory environments, code promotion

Q25	Puppet Forge - community modules, Puppetfile, r10k
Q26	Puppet node classification - ENC, site.pp, node definitions
Q27	Puppet resource ordering - require, before, notify, subscribe
Q28	Puppet templates - EPP vs ERB, template functions
Q29	Puppet Bolt - agentless task execution, plans, orchestration
Q30	Puppet PE (Enterprise) - console, RBAC, orchestrator, CD4PE
Q31	Chef at scale - Chef Automate, compliance, visibility, workflow
Q32	Puppet at scale - compiling catalogs, PuppetDB, performance
Q33	Migration from Chef/Puppet to Ansible - strategy, phased approach
Q34	Immutable infra vs config management - when Chef/Puppet is wrong
Q35	Chef cookbook converge fails - dependency conflict
Q36	Puppet agent not applying changes - certificate, catalog issues
Q37	Chef recipe runs but service not restarting - notification chain
Q38	Puppet drift - manual changes overwritten on next run
Q39	Chef/Puppet cookbook testing - catching errors before production
Q40	Large-scale migration: 500 nodes from Puppet to Ansible

SECTION 1 - Junior Core Concepts (Q1–Q15)

Q1 Chef basics - architecture, workstation, server, nodes

The Simple Answer

Chef is a configuration management tool that automates infrastructure setup and maintenance using Ruby-based code called recipes. Chef follows a pull-based, client-server architecture: the Chef Server stores cookbooks and node data, and Chef Client (agent) on each node periodically pulls its configuration and applies it.

The Analogy - A Restaurant Kitchen

The Chef Server is the head chef's recipe book in the kitchen office - it stores all recipes and knows which dish each table ordered. The Chef Workstation is where cooks write and test new recipes before adding them to the book. Each node (managed server) is a line cook with a copy of their assigned recipes - they check in with the head chef every 30 minutes, get their updated recipe list, and cook accordingly. If you change a recipe, every line cook picks it up on their next check-in.

Architecture Components

Chef Workstation - The developer's machine. Where you write, test, and upload cookbooks. Contains: knife (CLI tool), chef (development commands), Test Kitchen (testing), and Berkshelf (dependency management).

Chef Server - Central hub. Stores: cookbooks (the code), node objects (registered nodes and their attributes), data bags (shared data like passwords), environments (dev/staging/prod), and roles (named run-lists). Nodes authenticate via RSA key pairs.

Chef Client (Node) - Agent installed on every managed node. Runs periodically (default: every 30 minutes). Process: authenticate with server → build node object (gather system info via Ohai) → request run-list → download required cookbooks → compile and converge (apply the configuration) → report results back to server.

Ohai - System profiling tool (like Ansible's facts). Automatically collects: OS, IP, CPU, memory, disk, network, installed packages. Available as node attributes in recipes: `node['platform']`, `node['ipaddress']`, `node['memory']['total']`.

Q2 Chef resources - package, service, file, template

The Simple Answer

Resources are the building blocks of Chef recipes. Each resource describes the desired state of a piece of infrastructure. Chef ensures the system matches the desired state - if it already matches, no action is taken (idempotent).

Key Resources

package - Install or remove packages: `package 'nginx' do action :install end`. Chef auto-detects the package manager (apt on Debian, yum on RHEL).

service - Manage services: `service 'nginx' do action [:enable, :start] end`. Enable starts on boot. Start starts now.

file - Manage files: `file '/etc/motd' do content 'Welcome to production' owner 'root' mode '0644' end`.

template - Like file but processes ERB templates: template '/etc/nginx/nginx.conf' do source 'nginx.conf.erb' variables(port: 80, server_name: 'api.example.com') notifies :restart, 'service[nginx]' end.

directory - Create directories: directory '/opt/app' do owner 'app' mode '0755' recursive true end.

execute - Run a command: execute 'install_app' do command './install.sh' creates '/opt/app/bin/app' end. creates makes it idempotent - skip if the file already exists.

cookbook_file - Copy a static file from the cookbook's files/ directory to the node.

****The key principle:**** Resources describe WHAT the system should look like, not HOW to get there. "The nginx package should be installed" - Chef figures out whether to run apt-get or yum. This is declarative configuration.

Q3 Chef recipes and cookbooks

The Simple Answer

A recipe is a Ruby file (.rb) containing a list of resources to apply. A cookbook is a collection of related recipes, templates, files, attributes, and metadata - the fundamental unit of distribution in Chef.

Cookbook Structure

metadata.rb - Cookbook name, version, dependencies: depends 'apt', '~> 7.0'. recipes/default.rb - The default recipe (runs when the cookbook is included without specifying a recipe). recipes/install.rb - Additional recipes: include_recipe 'nginx::install'. templates/ - ERB templates for configuration files. files/ - Static files to copy to nodes. attributes/default.rb - Default attribute values. libraries/ - Custom Ruby code (helper functions, custom resources). spec/ and test/ - Unit and integration tests.

Recipe Execution

Recipes execute TOP-TO-BOTTOM. Resources are compiled into a resource collection, then converged (applied) in order. If a resource fails, the run stops (unless ignore_failure is set). The Chef Client reports success or failure back to the server.

Q4 Chef attributes - default, normal, override, precedence

The Simple Answer

Attributes customize cookbook behavior across different nodes, environments, and roles. The same cookbook installs Nginx with port 80 on web servers and port 8080 on API servers - controlled by attributes.

Attribute Types (lowest to highest precedence)

default - Set in cookbooks (attributes/default.rb). Lowest precedence. Meant to be overridden.

default['nginx']['port'] = 80.

normal - Set on the node object. Persists between runs. Survives cookbook changes. Used for node-specific values that should not reset.

override - Set in environments or roles. Higher than default. override['nginx']['port'] = 8443 in the production environment overrides the cookbook default.

automatic (Ohai) - System facts. Highest precedence. Cannot be overridden. `node['ipaddress'], node['platform'], node['memory']['total']`.

The Analogy

default attributes are the cookbook's suggestion: "Nginx should listen on port 80." override attributes are the environment's rule: "In production, Nginx listens on 443." automatic attributes are reality: "This server's IP is 10.0.1.5." Reality always wins over rules, which win over suggestions.

Q5 Chef templates - ERB

The Simple Answer

ERB (Embedded Ruby) templates generate configuration files with dynamic values. The template is processed by Chef, substituting variables and evaluating Ruby code, then the rendered file is placed on the node.

ERB Syntax

Output expression: `<%= @port %>` - Outputs the value of the variable. Code execution: `<% if @ssl_enabled %> ... <% end %>` - Conditional blocks. Iteration: `<% @upstream_servers.each do |server| %> server <%= server %>; <% end %>` - Loop over a list.

Example Template (nginx.conf.erb)

```
server { listen <%= @port %>; server_name <%= @server_name %>; <% if @ssl_enabled %> ssl_certificate
/etc/ssl/certs/<%= @domain %>.crt; <% end %> }
```

Template Resource

template '/etc/nginx/conf.d/app.conf' do source 'nginx.conf.erb' variables(port: node['nginx']['port'], server_name: node['nginx']['server_name'], ssl_enabled: node['nginx']['ssl'], domain: node['nginx']['domain']). notifies :reload, 'service[nginx]'. end. The notifies triggers an Nginx reload ONLY if the template content changed - if the file is identical, no reload occurs.

Q6 Chef run-list - ordering, roles

The Simple Answer

A run-list defines which recipes and roles are applied to a node, and in what order. A node's run-list might be: `["recipe[base]", "recipe[nginx]", "recipe[app-deploy]", "role[monitoring]"]`. Chef executes these in order - base setup first, then Nginx, then application deployment, then monitoring.

Roles

A role is a named run-list with attributes. Role "web-server": `run_list = ["recipe[nginx]", "recipe[ssl]", "recipe[app-deploy]"]`. `default_attributes: { nginx: { port: 80 } }`. Assign the role to 50 web servers - all get the same configuration. Change the role → all 50 servers update on their next run.

Environments

Environments separate configurations: development, staging, production. Each environment can pin cookbook versions: cookbook 'nginx', '= 5.0.0' in production (stability) while development uses the latest. Nodes belong to one environment.

Q7 Chef data bags - shared configuration

The Simple Answer

Data bags store global JSON data accessible to all nodes - database credentials, API keys, user lists, certificate data. Stored on the Chef Server. Accessed in recipes: `data_bag_item('users', 'admin')` returns the admin user's data.

Encrypted Data Bags

Encrypt sensitive data with a shared secret key. The data is stored encrypted on the Chef Server. Chef Client decrypts at runtime using the secret key stored on the node. Only the encrypted version is visible on the server - even Chef Server administrators cannot read the plaintext without the key.

Example

knife data bag create credentials database. Store: `{"id": "database", "username": "app_user", "password": "secret123"}`. In recipe: `db_creds = data_bag_item('credentials', 'database')`. Use: `db_creds['password']` to configure the application.

****Modern alternative:**** Use Chef Vault (simplified encrypted data bags with per-node access control) or integrate with HashiCorp Vault for runtime secret retrieval.

Q8 Chef knife - CLI tool

The Simple Answer

knife is Chef's command-line tool for interacting with the Chef Server. It manages: nodes, cookbooks, data bags, environments, roles, and more.

Essential Commands

knife bootstrap HOST - Install Chef Client on a new node and register it with the server. knife cookbook upload nginx - Upload a cookbook to the Chef Server. knife node list - List all registered nodes. knife node show web-1 - Show node details (run-list, attributes, platform). knife ssh "role:web-server" "sudo systemctl restart nginx" - Run a command on all web servers. knife data bag create secrets database - Create a data bag item. knife environment create production - Create an environment. knife status - Show all nodes with their last check-in time (find stale nodes).

Q9 Puppet basics - architecture, master, agent

The Simple Answer

Puppet is a configuration management tool that uses a declarative DSL (Puppet Language) to define system state. Like Chef, Puppet follows a pull-based, client-server architecture: the Puppet Server compiles a catalog of resources for each node, and the Puppet Agent on each node applies the catalog.

The Analogy - A School System

The Puppet Server is the school board - it has the curriculum (manifests) and knows which classes each student (node) needs. The Puppet Agent is the student - every 30 minutes, they check in with the school board, receive their personalized schedule (catalog), attend their classes (apply resources), and report their grades (report). If the curriculum changes, students pick up the new schedule on their next check-in.

Architecture Components

Puppet Server - Compiles catalogs from manifests. Written in Clojure/JRuby. Stores: manifests (Puppet code), modules (reusable code packages), Hiera data (configuration values), and node classifications.

Puppet Agent - Installed on every managed node. Runs every 30 minutes (configurable). Process: send facts (system info via Facter) to the server → server compiles a catalog (list of resources and their desired state) → agent applies the catalog → agent sends a report to the server.

PuppetDB - Database storing: facts from all nodes, catalogs, and reports. Enables: querying infrastructure state, finding all nodes with a specific fact, and dashboard reporting.

Facter - System profiling (like Chef's Ohai). Collects: OS, IP, hostname, CPU, memory, disk. Available as variables in manifests: `$facts['os']['family']`, `$facts['networking']['ip']`.

Hiera - Hierarchical data lookup. Separates data from code. See Q13.

Q10 Puppet resources - package, service, file

The Simple Answer

Resources are the fundamental building blocks of Puppet manifests. Each resource declares the desired state of a system element.

Key Resources

package - `package { 'nginx': ensure => installed }`. Or `ensure => '1.18.0'` for a specific version. Or `ensure => absent` to remove.

service - `service { 'nginx': ensure => running, enable => true }`. `Ensure running` starts it now. `Enable true` starts on boot.

file - `file { '/etc/nginx/nginx.conf': ensure => file, source => 'puppet:///modules/nginx/nginx.conf', owner => 'root', mode => '0644', notify => Service['nginx'] }`. `notify` triggers a service restart when the file changes.

user - `user { 'deploy': ensure => present, groups => ['docker', 'sudo'], shell => '/bin/bash', managehome => true }`.

exec - `exec { 'install_app': command => '/opt/install.sh', creates => '/opt/app/bin/app', path => ['/usr/bin', '/bin'] }`. `creates` makes it idempotent.

Resource Declaration Syntax

Puppet uses its own DSL (not Ruby like Chef, not YAML like Ansible): `resource_type { 'title': attribute => value, attribute => value, }`. The title uniquely identifies the resource. Attributes define the desired state.

Q11 Puppet manifests and modules

The Simple Answer

A manifest is a Puppet code file (.pp) containing resource declarations and logic. A module is a directory containing related manifests, templates, files, data, and tests - the fundamental unit of reusable Puppet code.

Module Structure

manifests/init.pp - The main class (class name matches the module name). manifests/install.pp - class nginx::install { package { 'nginx': ensure => installed } }. manifests/config.pp - Configuration management. manifests/service.pp - Service management. templates/ - ERB or EPP templates. files/ - Static files served to nodes. data/ - Hieradata specific to this module. spec/ - Unit tests (rspec-puppet). metadata.json - Module name, version, dependencies, supported OS.

The init-install-config-service Pattern

Most modules follow this pattern: init.pp includes install, config, and service classes in order. install.pp installs packages. config.pp deploys configuration files. service.pp manages the service. Each class is small and focused. Dependencies ensure order: Class['nginx::install'] -> Class['nginx::config'] ~> Class['nginx::service'].

Q12 Puppet classes and defined types

The Simple Answer

Classes are named blocks of Puppet code that are applied once per node. class nginx { package { 'nginx': ensure => installed } }. Include a class: include nginx or class { 'nginx': port => 8080 }. A class is a singleton - including it multiple times has no effect (it runs once).

Defined types are reusable resource patterns that CAN be applied multiple times. define nginx::vhost(String \$server_name, Integer \$port = 80) { file { ["/etc/nginx/conf.d/\${name}.conf": content => template('nginx/vhost.conf.erb')] }. Use multiple times: nginx::vhost { 'api': server_name => 'api.example.com' }. nginx::vhost { 'web': server_name => 'www.example.com', port => 443 }.

When to Use Which

Class: one-time configuration (install Nginx, configure SSH, set up monitoring). Defined type: repeated patterns (multiple virtual hosts, multiple database users, multiple cron jobs).

Q13 Puppet Hiera - external data, hierarchy

The Simple Answer

Hiera separates data from code. Instead of hardcoding values in manifests, Puppet looks up values in Hiera's hierarchical data files. The hierarchy searches from most specific to least specific, returning the first match.

Hierarchy Example (hiera.yaml)

hierarchy: - name: "Per-node" path: "nodes/{facts.networking.fqdn}.yaml" - name: "Per-environment" path: "environments/{environment}.yaml" - name: "Per-OS" path: "os/{facts.os.family}.yaml" - name: "Common" path: "common.yaml".

How It Works

Puppet needs the value of nginx::port. Hiera searches: nodes/web-1.example.com.yaml → environments/production.yaml → os/Debian.yaml → common.yaml. First file containing nginx::port wins. This allows: common defaults (common.yaml: nginx::port: 80), environment overrides (production.yaml: nginx::port: 443), and node-specific values (web-1.yaml: nginx::port: 8080).

The Analogy

Hiera is like asking a question up a chain of command. "What port should Nginx use?" Ask the node-specific config first (the server itself). No answer? Ask the environment config (production). No answer? Ask the OS-specific config. Still nothing? Ask the common config (the default). The first answer you get is the one you use.

Q14 Puppet Facter - system facts

The Simple Answer

Facter collects system information and makes it available as variables in Puppet manifests. Run `facter` on any Puppet-managed node to see all facts.

Common Facts

`$facts['os']['family']` - Debian, RedHat, windows. `$facts['os']['release']['major']` - 22 (Ubuntu 22.04).

`$facts['networking']['ip']` - Primary IP address. `$facts['processors']['count']` - CPU count.

`$facts['memory']['system']['total']` - Total RAM. `$facts['kernel']` - Linux, windows.

Conditional Logic with Facts

`if $facts['os']['family'] == 'Debian' { package { 'nginx': ensure => installed, provider => apt } } elsif $facts['os']['family'] == 'RedHat' { package { 'nginx': ensure => installed, provider => yum } }`. Write one module that works on any Linux distribution by branching on facts.

Custom Facts

Write custom facts in Ruby: `Facter.add('app_version') do setcode do Facter::Util::Resolution.exec('cat /opt/app/VERSION') end end`. Custom facts report application-specific information back to the Puppet Server, enabling: inventory tracking, conditional logic based on deployed app versions, and PuppetDB queries.

Q15 Chef vs Puppet vs Ansible - comparison

The Simple Answer

Feature	Chef	Puppet	Ansible
Language	Ruby DSL	Puppet DSL	YAML
Architecture	Client-server (pull)	Client-server (pull)	Agentless (push)
Agent required	Yes (Chef Client)	Yes (Puppet Agent)	No (SSH)
Server required	Yes (Chef Server)	Yes (Puppet Server)	No (control node)
Learning curve	High (Ruby)	Medium (custom DSL)	Low (YAML)
Idempotency	Built-in (resources)	Built-in (resources)	Built-in (modules)
State enforcement	Continuous (30 min)	Continuous (30 min)	On-demand (push)

Community	Declining	Declining	Largest, growing
Best for	Complex Ruby shops	Enterprise compliance	Multi-purpose, cloud
Testing	Test Kitchen, ChefSpec	rspec-puppet, Beaker	Molecule, Testinfra

The Honest Assessment

Ansible has won the configuration management market. Its agentless architecture, YAML simplicity, and massive community make it the default choice for new projects. Chef and Puppet are found in: legacy environments (organizations that adopted them 5-10 years ago), enterprises with heavy compliance requirements (Puppet's reporting and Chef InSpec excel here), and Ruby-centric organizations. For interviews: know Chef/Puppet concepts to work with existing codebases, but recommend Ansible for new implementations.

SECTION 2 - Mid-Level Production Patterns (Q16–Q30)

Q16 Chef environments - dev/staging/prod

The Simple Answer

Environments separate configurations and pin cookbook versions. The production environment pins: cookbook 'nginx', '= 5.0.0' - preventing untested cookbook versions from reaching production. Development uses the latest: cookbook 'nginx', '>= 0.0.0'. Cookbook promotion workflow: develop → upload to server → test in development environment → pin version in staging → after validation, pin in production.

Environment Attributes

Each environment can override default attributes: production overrides default['app']['log_level'] = 'warn' (less verbose logging). Development keeps default['app']['log_level'] = 'debug'. Same cookbook, different behavior per environment - controlled by environment attributes.

Q17 Chef roles - abstraction, run-lists

The Simple Answer

Roles group recipes and attributes for a server function. Role "web-server": run_list ["recipe[base]", "recipe[nginx]", "recipe[ssl]", "recipe[monitoring]"]. Role "database": run_list ["recipe[base]", "recipe[postgresql]", "recipe[backup]", "recipe[monitoring]"]. Assign a role to a node instead of individual recipes. Change the role → all nodes with that role update.

The Problem with Roles

Roles are stored on the Chef Server, NOT in version control. Changes to roles are immediate and global - no code review, no testing, no rollback. A mistake in a role affects every node with that role instantly. This is why Chef introduced Policyfiles (Q21) as the modern replacement.

Q18 Chef Berkshelf - dependency management

The Simple Answer

Berkshelf manages cookbook dependencies. The Berksfile lists dependencies: source 'https://supermarket.chef.io'. cookbook 'nginx', '~> 12.0'. cookbook 'app', path: '../app-cookbook'. berks install downloads dependencies. berks upload pushes to the Chef Server. berks vendor vendors all dependencies into a local directory for air-gapped environments.

Version Constraints

'~> 12.0' - Any version 12.x (pessimistic constraint). '>= 5.0', '< 6.0' - Between 5.0 and 6.0. '= 5.2.1' - Exact version. Always pin versions in production. Unpinned dependencies change unexpectedly and break runs.

Q19 Chef InSpec - compliance as code

The Simple Answer

InSpec tests whether a system meets compliance requirements. Unlike Chef recipes (which configure systems), InSpec profiles verify that systems ARE configured correctly. InSpec is: agentless (runs over SSH/WinRM), human-readable (describe syntax), and audit-ready (produces compliance reports).

Example

describe package('nginx') do it { should be_installed } end. describe port(80) do it { should be_listening } end. describe file('/etc/nginx/nginx.conf') do it { should be_file } its('mode') { should cmp '0644' } its('owner') { should eq 'root' } end.

CIS Benchmarks

InSpec profiles exist for CIS benchmarks (Center for Internet Security). Run: `inspec exec cis-benchmark-profile` against any server. Produces a compliance report: 82 of 95 controls passed. 13 failures with remediation guidance. Auditors accept InSpec reports as compliance evidence.

****InSpec is Chef's lasting contribution.**** Even organizations migrating away from Chef keep InSpec - it is the best compliance testing tool available, independent of which configuration management tool you use.

Q20 Chef Test Kitchen - testing cookbooks

The Simple Answer

Test Kitchen creates disposable environments (Docker containers, Vagrant VMs, cloud instances), applies your cookbook, and runs verification tests. The workflow: `kitchen create` (spin up instance) → `kitchen converge` (apply cookbook) → `kitchen verify` (run InSpec tests) → `kitchen destroy` (clean up). Run on every cookbook change - catch errors before they reach production.

.kitchen.yml

`driver: docker` (or `vagrant`, `ec2`, `gce`). `provisioner: chef_zero` (runs Chef locally without a server). `platforms: [ubuntu-22.04, centos-8]` (test on multiple OS). `suites: [{name: default, run_list: ["recipe[nginx]"], verifier: {name: inspec}}]`. This tests the nginx cookbook on both Ubuntu and CentOS, verifying with InSpec.

Q21 Chef Policyfile - modern workflow

The Simple Answer

Policyfiles replace roles and environments with a single, versioned, testable artifact. A `Policyfile.rb` defines: `run_list ["recipe[base]", "recipe[nginx]", "recipe[app]"]`. `cookbook 'nginx', '~> 12.0'`. `default['nginx']['port'] = 80`. `chef install` compiles a `Policyfile.lock.json` (pinned versions - like `package-lock.json`). `chef push production` uploads the policy. Every aspect of the node's configuration is: version-controlled, tested as a unit, and promoted through environments as a single artifact.

Why Policyfiles Are Better

Roles are mutable and global - a role change affects all nodes immediately with no review. Policyfiles are immutable and versioned - push a new version to staging, test, then push to production. The same deterministic configuration is guaranteed across environments. Policyfiles are the recommended Chef workflow since 2016.

Q22 Chef Habitat - application automation

The Simple Answer

Habitat packages applications with their runtime dependencies into a self-contained artifact that runs anywhere - bare metal, VMs, containers, or Kubernetes. A `Habitat plan.sh` defines: how to build the application, its runtime

dependencies, configuration templates, and health checks. The Habitat Supervisor manages the running application: applies configuration updates, performs health checks, and handles service discovery.

Why Habitat

Habitat decouples the application from the infrastructure. The same Habitat package runs identically on: a Docker container, a Kubernetes Pod, a bare metal server, or a cloud VM. No Chef Server needed - the Supervisor is autonomous. Habitat competes with Docker + Helm in the container world but works on non-containerized infrastructure too.

Q23 Chef Infra Client - solo and zero mode

The Simple Answer

chef-solo - Runs Chef without a Chef Server. Cookbooks are local (no server upload needed). Used for: local development, simple single-server setups, and environments where a Chef Server is overkill.

chef-zero (local mode) - Starts an in-memory Chef Server, runs the cookbook, and shuts down. chef-client --local-mode. Used for: testing cookbooks locally, CI/CD pipelines, and development. Behaves identically to a real Chef Server run but without the server infrastructure.

Q24 Puppet environments - directory environments

The Simple Answer

Puppet environments separate code versions. Each environment is a directory under /etc/puppetlabs/code/environments/: production/, staging/, development/. Each contains: manifests/, modules/, data/ (Hiera), and environment.conf. Nodes are assigned to an environment (default: production). The Puppet Server serves different code to different environments.

Code Promotion Pattern

Develop in a feature branch → merge to development branch → auto-deploy to development environment → test → merge to staging → test → merge to production. Git branches map to Puppet environments. r10k or Code Manager (PE) deploys code from Git to the Puppet Server automatically.

Q25 Puppet Forge - community modules, r10k

The Simple Answer

Puppet Forge (forge.puppet.com) is the community module repository - like Ansible Galaxy or Helm's Artifact Hub. Thousands of modules for: nginx, apache, mysql, postgresql, docker, kubernetes, monitoring, and more. Puppetfile lists dependencies: mod 'puppetlabs/nginx', '4.0.0'. mod 'puppetlabs/stdlib', '8.0.0'.

r10k

r10k deploys Puppet code from Git to the Puppet Server. It reads the Puppetfile, downloads Forge modules, and deploys everything to the correct environment directory. Workflow: push code to Git → r10k deploy environment production → Puppet Server has the latest code. Automate with webhooks: Git push → webhook triggers r10k deploy. Code Manager in Puppet Enterprise provides the same functionality with a GUI.

Q26 Puppet node classification - ENC, site.pp

The Simple Answer

Node classification determines which classes are applied to each node. Three approaches:

site.pp - Direct node definitions: node 'web-1.example.com' { include nginx; include monitoring }. Simple but does not scale - one entry per node.

Hiera with automatic lookup - Use hiera data to assign classes: in nodes/web-1.yaml: classes: ['nginx', 'monitoring']. In site.pp: lookup('classes', Array, 'unique').include. Scales better - data-driven classification.

External Node Classifier (ENC) - A script or service that returns YAML with classes and parameters for each node. Puppet calls the ENC with the node name. The ENC returns: classes, parameters, and environment. Used for: dynamic classification from a CMDB, cloud API, or custom database. Puppet Enterprise Console is an ENC with a web GUI.

Q27 Puppet resource ordering - require, before, notify

The Simple Answer

Puppet does NOT apply resources in the order they appear in the manifest. By default, Puppet applies resources in an arbitrary order determined by the catalog compiler. You must explicitly declare dependencies.

Ordering Metaparameters

require - "Apply this AFTER that." Service['nginx'] { require => Package['nginx'] }. Install package before starting service.

before - "Apply this BEFORE that." Package['nginx'] { before => Service['nginx'] }. Same relationship, declared on the other resource.

notify - "Apply this AFTER that, AND trigger a refresh if it changes." File['/etc/nginx/nginx.conf'] { notify => Service['nginx'] }. If the config file changes, restart Nginx.

subscribe - "Refresh this when that changes." Service['nginx'] { subscribe => File['/etc/nginx/nginx.conf'] }. Same as notify but declared on the receiving end.

Chaining Arrows

Package['nginx'] -> File['/etc/nginx/nginx.conf'] ~> Service['nginx']. -> means "before." ~> means "notify (before + trigger refresh)." Read left to right: install package, then configure file, then manage service (and restart if config changed).

****The critical difference from Chef and Ansible:**** Chef and Ansible execute in order of declaration (top to bottom). Puppet requires explicit ordering. This catches beginners - "I put Package before Service in my manifest, why does Service run first?" Because Puppet does not care about declaration order unless you specify dependencies.

Q28 Puppet templates - EPP vs ERB

The Simple Answer

ERB (Embedded Ruby) - Legacy. Uses Ruby syntax: <%= @port %> for output, <% if @ssl %> for logic. Same as Chef templates. Still widely used in existing Puppet code.

EPP (Embedded Puppet) - Modern. Uses Puppet syntax: <%= \$port %>, <% if \$ssl { %> ... <% } %>. Type-safe. Better error messages. The recommended format for new Puppet code.

Using Templates

In a manifest: `file { '/etc/nginx/nginx.conf': content => epp('nginx/nginx.conf.epp', { port => $port, ssl => $ssl }) }`. Or ERB: `content => template('nginx/nginx.conf.erb')`.

Q29 Puppet Bolt - agentless execution

The Simple Answer

Bolt is Puppet's agentless task execution tool - similar to Ansible's ad-hoc commands. It connects via SSH or WinRM and runs commands, scripts, or Puppet code on remote nodes WITHOUT requiring the Puppet Agent.

Key Features

Tasks - Single scripts (Bash, PowerShell, Python, Ruby) packaged with metadata. `bolt task run mymodule::restart_service --targets web-servers`. Plans - Multi-step orchestration workflows written in Puppet Language or YAML. A plan can: run tasks, apply Puppet manifests, make decisions based on results, and run in parallel or sequentially. Inventory - Groups of targets with connection info. Supports: static YAML, PuppetDB queries, and cloud provider dynamic inventory.

Why Bolt Exists

Puppet's pull model (agent checks every 30 minutes) is slow for immediate operations. Bolt fills the gap - run commands NOW across 100 nodes. It bridges Puppet's configuration management with Ansible-style ad-hoc execution. Used for: emergency patches, one-time operations, orchestrated deployments, and bootstrapping new nodes.

Q30 Puppet Enterprise - console, RBAC, orchestrator

The Simple Answer

Puppet Enterprise (PE) adds commercial features on top of open-source Puppet.

Console - Web GUI for: node management, report viewing, classification, and real-time infrastructure overview. Non-technical users can see: how many nodes are compliant, which nodes failed their last run, and the distribution of operating systems.

RBAC - Role-based access control for the console and API. Restrict who can: deploy code, classify nodes, view reports, and manage settings.

Orchestrator - Run Puppet on specific nodes on-demand (instead of waiting for the 30-minute interval). Useful for: immediate deployments, emergency changes, and orchestrated rolling updates.

Code Manager - Automated code deployment from Git. Push to Git → Code Manager deploys to the Puppet Server. Replaces manual r10k deployment.

CD4PE (Continuous Delivery for PE) - CI/CD pipeline specifically for Puppet code. Test Puppet code changes before deploying to production. Similar to how application CI/CD works but for infrastructure code.

SECTION 3 - Senior Architecture (Q31–Q34)

Q31 Chef at scale - Chef Automate

The Simple Answer

Chef Automate is the enterprise dashboard for Chef infrastructure. It provides: convergence status (which nodes succeeded/failed), compliance scanning (InSpec reports across all nodes), infrastructure visibility (dashboard of all nodes, platforms, cookbooks), and workflow pipelines (CI/CD for cookbooks).

Scaling Challenges

Chef Server performance - At 5000+ nodes, the Chef Server's PostgreSQL database becomes a bottleneck. Node check-ins create significant read/write load. Solutions: tune PostgreSQL, add read replicas, stagger node check-in intervals (splay).

Cookbook complexity - Large organizations accumulate hundreds of cookbooks with complex dependencies. Berkshelf dependency resolution becomes slow. Solutions: Policyfiles (compile dependencies once, deploy the lock file), minimize cross-cookbook dependencies, deprecate unused cookbooks.

Compliance at scale - Running InSpec profiles across 10,000 nodes generates massive report data. Chef Automate aggregates and trends compliance data. Dashboard shows: 85% of nodes meet CIS Level 1, 12% have critical failures, 3% unreachable.

Q32 Puppet at scale - PuppetDB, catalog compilation

The Simple Answer

Catalog compilation is the primary bottleneck. Each node receives a unique catalog - the Puppet Server compiles it from manifests, Hiera data, and facts. At 5000+ nodes, compilation consumes significant CPU and memory.

Scaling Solutions

Compile masters - Multiple Puppet Servers behind a load balancer. The CA (Certificate Authority) master handles certificates. Compile masters handle catalog compilation. Scale horizontally by adding compile masters.

PuppetDB tuning - PuppetDB stores facts, catalogs, and reports. PostgreSQL backend. At scale: tune connection pools, add read replicas, configure fact expiration, and archive old reports.

Catalog caching - If a node's facts have not changed, serve the cached catalog instead of recompiling. Significant performance improvement for stable infrastructure.

Environment caching - Cache parsed manifests in memory. The Puppet Server does not re-parse files for every compilation. Clear cache after code deployment.

Q33 Migration from Chef/Puppet to Ansible

The Strategy

Phase 1 - Inventory and assess (2-4 weeks). List all cookbooks/modules. Map each to an Ansible equivalent: existing Ansible Galaxy role, or needs to be written. Categorize by: criticality (critical infrastructure vs nice-to-have), complexity (simple package install vs complex multi-service orchestration), and dependencies (standalone vs deeply integrated).

Phase 2 - Parallel run (ongoing). New servers are configured with Ansible from day one. Existing servers continue with Chef/Puppet. Both tools coexist. No rush to migrate existing infrastructure.

Phase 3 - Incremental migration (3-12 months). Convert one cookbook/module at a time, starting with: simplest and least critical first. For each conversion: write Ansible role → test with Molecule → deploy to staging → validate → deploy to production → remove the corresponding Chef cookbook/Puppet module from the node. Gradually shrink the Chef/Puppet footprint.

Phase 4 - Decommission (1-2 months). After all configurations are migrated: remove Chef Client/Puppet Agent from all nodes. Decommission the Chef Server/Puppet Server. Archive cookbooks/modules for reference.

Common Pitfalls

Underestimating effort - A 200-cookbook Chef codebase takes 6-12 months to migrate fully. Plan realistically. Losing continuous enforcement - Chef/Puppet agents enforce state every 30 minutes. Ansible only enforces when run.

Implement: ansible-pull with cron, or AWX scheduled jobs to match the continuous enforcement model.

Variable/data migration - Chef attributes/data bags and Puppet Hiera do not map 1:1 to Ansible variables. Plan the variable hierarchy carefully.

Q34 Immutable infra vs config management

The Simple Answer

Configuration management (Chef/Puppet/Ansible) - Mutate existing servers. Install packages, update configs, restart services. Servers accumulate state over time. Drift is possible (and detected by the next agent run).

Immutable infrastructure - Never modify running servers. Build a new image (AMI, VM image) with all changes baked in. Deploy by replacing servers (ASG rolling update). Old servers are terminated. No drift is possible - every server is a fresh image.

When Chef/Puppet Is Wrong

If you build AMIs with Packer and deploy via Auto Scaling Groups with zero in-place changes, running Chef/Puppet agents on the instances adds: complexity (agent configuration, server maintenance), cost (Chef/Puppet Server infrastructure), and risk (agent failures, configuration drift between image builds). In this model, Chef/Puppet adds overhead without value.

When Chef/Puppet Is Right

Long-lived servers that cannot be easily replaced (databases, stateful services). Compliance environments requiring continuous state enforcement and reporting (Chef InSpec, Puppet compliance modules). Organizations with existing Chef/Puppet investment and no business case for migration. Network device management (some Puppet modules manage switches and routers).

****The pragmatic answer:**** For new infrastructure: use Ansible for configuration + Packer for image building + Terraform for deployment. For existing Chef/Puppet shops: continue using them if they work well. Migrate only when there is a clear business benefit (cost reduction, skill availability, simplification). Do not migrate for the sake of being "modern."

SECTION 4 - Tricky Scenarios (Q35–Q40)

Q35 Chef cookbook converge fails - dependency conflict

What Happened

Running chef-client fails with: "Unable to satisfy constraints on cookbook 'apt': apt >= 7.0 required by 'nginx', apt < 7.0 required by 'legacy-app'." Two cookbooks require incompatible versions of the same dependency.

Step-by-Step

Step 1 - Identify the conflict. Read the error carefully. Which cookbooks have conflicting constraints? Check each cookbook's metadata.rb for depends statements.

Step 2 - Check for updates. Is there a newer version of either cookbook that has relaxed its constraint? Check Chef Supermarket for updates. berks update may resolve the conflict if compatible versions exist.

Step 3 - If no compatible version exists. Fork one of the conflicting cookbooks and update its dependency constraint. Or: upgrade the legacy-app cookbook to work with the newer apt version (the proper fix but more work).

Step 4 - Use Policyfiles. Policyfiles resolve dependencies at compile time and produce a lock file. If a conflict exists, it fails during policy compilation (on your workstation) - not during production convergence. This catches dependency issues before they reach any node.

Prevention

Pin all dependency versions in Berksfile with pessimistic constraints (~> X.Y). Test dependency resolution in CI before uploading to the Chef Server. Use Policyfiles for deterministic dependency resolution.

Q36 Puppet agent not applying changes

What Happened

You updated a Puppet module and deployed to the Puppet Server. Nodes are not picking up the changes - they still have the old configuration after multiple agent runs.

Step-by-Step

Step 1 - Check code deployment. Did r10k or Code Manager actually deploy the code? Check on the Puppet Server: ls /etc/puppetlabs/code/environments/production/modules/nginx/ - is the updated code there?

Step 2 - Check environment caching. The Puppet Server caches parsed manifests. If environment caching is enabled, the server may serve the old code. Clear cache: service puppetserver restart or use the environment_cache API endpoint.

Step 3 - Check the node's environment. Is the node in the correct environment? puppet agent --configprint environment on the node. If it is in "development" but you deployed to "production," the node gets the old code.

Step 4 - Check certificate issues. If the agent's certificate was revoked or expired, the agent cannot communicate with the server. Check: puppet agent -t --debug shows SSL errors. Fix: puppetserver ca clean node-name and re-sign.

Step 5 - Check the catalog. Run puppet agent -t --noop on the node. If it shows no changes, the catalog compilation is not including your class. Check: node classification (site.pp, ENC, Hiera).

Q37 Chef recipe runs but service not restarting

What Happened

A Chef template resource updates the Nginx configuration file. The file changes on the node. But Nginx is not restarted - it is serving the old configuration.

Why It Happens

The template resource does not have a notifies statement to restart the service. Or: the notifies references the wrong service name. Or: the service resource has action :nothing (it only runs when notified) but the notification chain is broken.

Step-by-Step

Step 1 - Check the template resource. Does it have: notifies :restart, 'service[nginx]'? The service name must match exactly - 'service[nginx]' not 'service[Nginx]' (case-sensitive).

Step 2 - Check the service resource. Is it defined? service 'nginx' do action [:enable, :start] end. If it has action :nothing, it ONLY runs when notified - ensure the notification source is correct.

Step 3 - Check the converge output. Run chef-client in debug mode. Look for: "template[/etc/nginx/nginx.conf] updated file" (the file changed) followed by "service[nginx] restarted" (the notification triggered). If the file shows "up to date," the template content is identical - no change, no notification.

Step 4 - Delayed vs immediate. By default, notifies is :delayed - runs at the END of the converge. If another resource fails before the end, the notification is skipped. Use notifies :immediately, 'service[nginx]' for critical restarts.

Q38 Puppet drift - manual changes overwritten

What Happened

A developer SSHed into a production server and manually changed /etc/nginx/nginx.conf to fix an urgent issue. 30 minutes later, the Puppet agent ran and reverted the change to the version in the Puppet module. The fix was lost. The issue returned.

Why This Is Correct Behavior

Puppet's job is to enforce the desired state defined in manifests. A manual change is drift - Puppet corrects it. This is a feature, not a bug. The problem is not Puppet reverting the change - the problem is that the fix was applied manually instead of through Puppet.

Step-by-Step

Step 1 - Immediate fix. Re-apply the manual change (again). Disable the Puppet agent on this node temporarily: puppet agent --disable "Emergency fix - JIRA-789". This prevents the agent from reverting the change until the proper fix is in Puppet.

Step 2 - Proper fix. Update the Puppet module with the fix. Push to Git. Deploy to the Puppet Server. Re-enable the agent: puppet agent --enable.

Step 3 - Cultural fix. Establish a rule: no manual changes to Puppet-managed nodes. All changes go through Puppet code → Git → code review → deploy. If a truly urgent fix is needed, disable the agent first, apply the fix, then commit the change to Puppet within 24 hours.

****The lesson:**** Configuration management's value is enforcing desired state. If you allow manual changes to persist, you lose the guarantee that all nodes match the code. Puppet's "overwrite" behavior is intentional and critical for maintaining consistency across hundreds of nodes.

Q39 Chef/Puppet cookbook testing

Step-by-Step

For Chef:

ChefSpec (unit tests) - Test recipe logic without converging. describe 'nginx::default' do let(:chef_run) { ChefSpec::SoloRunner.new.converge(described_recipe) }; it 'installs nginx' do expect(chef_run).to install_package('nginx') end; it 'starts the service' do expect(chef_run).to start_service('nginx') end; end. Fast (milliseconds). Tests logic, not actual system state.

Test Kitchen (integration tests) - Converge the cookbook on a real instance (Docker container) and verify with InSpec. kitchen test runs: create → converge → verify → destroy. Catches: incorrect package names, template rendering errors, service management issues, and dependency conflicts.

CI Pipeline: Lint (cookstyle) → Unit test (ChefSpec) → Integration test (Test Kitchen) → Upload to Chef Server. Every cookbook change goes through this pipeline.

For Puppet:

rspec-puppet (unit tests) - Test catalog compilation. it { is_expected.to contain_package('nginx').with_ensure('installed') }. Tests that the catalog contains the expected resources without applying them.

Beaker / Litmus (integration tests) - Apply the module on a real instance and verify. Puppet Litmus is the modern replacement for Beaker. Uses Docker or cloud instances. Runs: apply manifest → verify with InSpec or ServerSpec.

CI Pipeline: Lint (puppet-lint) → Syntax check (puppet parser validate) → Unit test (rspec-puppet) → Integration test (Litmus) → Deploy to Puppet Server.

Q40 Large-scale migration: 500 nodes from Puppet to Ansible

The Complete Strategy

Phase 1 - Assessment (3 weeks). Inventory all Puppet modules: 45 modules, 12 custom facts, 8 defined types, 300+ Hiera data files. Categorize: 15 simple modules (package + config + service), 20 medium modules (with templates and conditions), 10 complex modules (custom types, external data lookups). Identify dependencies: which modules depend on which. Map to Ansible: 30 have direct Galaxy role equivalents, 15 need custom Ansible roles.

Phase 2 - Foundation (2 weeks). Set up Ansible infrastructure: AWX/Tower for centralized management. Dynamic inventory from the same sources Puppet uses. group_vars structure mirroring Hiera hierarchy. Vault for secrets (replacing encrypted Hiera).

Phase 3 - Pilot (4 weeks). Migrate 5 non-critical modules (the simplest). Write Ansible roles. Test with Molecule. Deploy to 20 non-production nodes. Run Puppet and Ansible in parallel for 2 weeks - both managing the same nodes. Compare: do they produce the same result? Fix discrepancies.

Phase 4 - Incremental migration (6-9 months). Migrate one module per week, prioritized by: simplicity first, then medium, then complex. For each module: write Ansible role → test → deploy to staging → validate → deploy to production → remove the Puppet module from affected nodes' classification.

Phase 5 - Continuous enforcement (parallel with Phase 4). Configure ansible-pull or AWX scheduled jobs to run every 30 minutes - matching Puppet's enforcement frequency. Without this, you lose continuous state enforcement.

Phase 6 - Decommission (4 weeks). After all modules migrated and stable for 30 days: remove Puppet Agent from all nodes. Decommission Puppet Server and PuppetDB. Archive Puppet code for reference. Update documentation.

Key Metrics

Track: migration progress (modules migrated / total), parallel validation results (Puppet vs Ansible match rate), node coverage (% of nodes fully on Ansible), incident rate (any incidents caused by migration), and enforcement frequency (are AWX jobs running on schedule).

Budget

500 nodes, 45 modules: estimated 6-9 months with 2 engineers dedicated. The largest cost is not the Ansible writing - it is testing, validation, and the parallel run period. Do not rush the migration - a botched migration causes more damage than running Puppet for another year.

Interview Tips

For Junior Roles

Focus on Q1-Q15. Know the architecture of both Chef and Puppet (client-server, pull-based, agent). Explain resources (package, service, file). Understand Chef vs Puppet vs Ansible trade-offs. Be able to write a simple recipe or manifest.

For Mid-Level Roles

Q1-Q30. Be ready to discuss: Chef environments and Policyfiles, Puppet Hiera and r10k, InSpec for compliance, Test Kitchen and rspec-puppet for testing, and Puppet Bolt for ad-hoc execution. Know why Policyfiles replaced roles.

For Senior Roles

All 40 questions. Migration strategy, scaling challenges, and the decision between config management vs immutable infrastructure separate senior from mid-level. Be ready to plan a large-scale migration (Q40).

Quick Reference Cheat Sheet

Topic	Key Takeaway
Chef architecture	Workstation → Server → Client (agent). Pull every 30 min. Ruby DSL.
Chef resources	package, service, file, template, execute. Declarative desired state.
Chef cookbooks	recipes/ + templates/ + attributes/ + files/ + metadata.rb. Unit of distribution.
Chef attributes	default (lowest) → normal → override → automatic/Ohai (highest).
Chef data bags	Global JSON data. Encrypted for secrets. Access with <code>data_bag_item()</code> .
Chef Policyfiles	Modern replacement for roles+environments. Versioned, deterministic, testable.
Chef InSpec	Compliance as code. Agentless. CIS benchmarks. Audit-ready reports.
Chef Test Kitchen	Create → converge → verify → destroy. Docker or cloud. InSpec verification.
Puppet architecture	Server compiles catalog → Agent applies catalog. Pull every 30 min. Puppet DSL.
Puppet resources	package, service, file, exec, user. Must declare ordering explicitly.
Puppet modules	manifests/ + templates/ + files/ + data/ + metadata.json. init-install-config-service pattern.
Puppet Hiera	Hierarchical data lookup. Separates data from code. Per-node to common fallback.
Puppet Facter	System facts. Used in conditionals. Custom facts for app-specific data.
Puppet ordering	NOT in declaration order. Use <code>require/before/notify/subscribe</code> or <code>-></code> / <code>~></code> arrows.
Puppet Bolt	Agentless ad-hoc execution. Tasks and plans. SSH/WinRM. Like Ansible's ad-hoc.
Puppet r10k	Deploy Puppet code from Git. Puppetfile for Forge modules. Automate with webhooks.
Chef vs Puppet vs Ansible	Ansible won. Chef/Puppet for legacy and compliance. Ansible for new projects.
Migration strategy	Assess → parallel run → incremental migration → decommission. 6-12 months.
Immutable vs config mgmt	Immutable for stateless (containers, ASGs). Config mgmt for stateful and compliance.
Drift handling	Puppet/Chef CORRECT drift by design. Manual changes are reverted. This is the feature.